

Part 1 基础概念

1.3 程序地址

程序各阶段的地址变化

1.源程序编译后的main.s文件

add: 符号表示

符号地址

```
main:
.LFB0:
    .cfi_startproc
    pushq    %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movq     %rsp, %rbp
    .cfi_def_cfa_register 6
    movl     $13, %esi
    movl     $20, %edi
    call     add
    popq     %rbp
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc
```

Part 1 基础概念

1.3 程序地址

程序各阶段的地址变化

2.源程序汇编后
的main.o和
test.o反汇编后
的文件

13: 整数值
相对地址

```
0000000000000000 <main>:
0: 55                                push    %rbp
1: 48 89 e5                          mov     %rsp,%rbp
4: be 0d 00 00 00                    mov     $0xd,%esi
9: bf 14 00 00 00                    mov     $0x14,%edi
e: e8 00 00 00 00                    callq   13 <main+0x13>
13: 5d                                pop     %rbp
14: c3                                retq

0000000000000000 <add>:
0: 55                                push    %rbp
1: 48 89 e5                          mov     %rsp,%rbp
4: 89 7d ec                          mov     %edi,-0x14(%rbp)
7: 89 75 e8                          mov     %esi,-0x18(%rbp)
a: 8b 45 e8                          mov     -0x18(%rbp),%eax
d: 8b 55 ec                          mov     -0x14(%rbp),%edx
10: 01 d0                            add     %edx,%eax
12: 89 45 fc                          mov     %eax,-0x4(%rbp)
15: 8b 45 fc                          mov     -0x4(%rbp),%eax
18: 5d                                pop     %rbp
19: c3                                retq
```

Part 1 基础概念

1.3 程序地址

程序各阶段的地址变化

3.可执行程序myprog反汇编后 4004e2: 整数值

相对地址

```
00000000004004cd <main>:
4004cd: 55                push    %rbp
4004ce: 48 89 e5          mov     %rsp,%rbp
4004d1: be 0d 00 00 00    mov     $0xd,%esi
4004d6: bf 14 00 00 00    mov     $0x14,%edi
4004db: e8 02 00 00 00    callq  4004e2 <add>
4004e0: 5d                pop     %rbp
4004e1: c3                retq

00000000004004e2 <add>:
4004e2: 55                push    %rbp
4004e3: 48 89 e5          mov     %rsp,%rbp
4004e6: 89 7d ec          mov     %edi,-0x14(%rbp)
4004e9: 89 75 e8          mov     %esi,-0x18(%rbp)
4004ec: 8b 45 e8          mov     -0x18(%rbp),%eax
4004ef: 8b 55 ec          mov     -0x14(%rbp),%edx
4004f2: 01 d0             add     %edx,%eax
4004f4: 89 45 fc          mov     %eax,-0x4(%rbp)
4004f7: 8b 45 fc          mov     -0x4(%rbp),%eax
```

1.3 程序地址

- 程序各阶段的地址变化

4. 可执行程序装入内存执行时的地址？

内存单元地址

CPU对内存
的访问方式
决定

物理地址

Part 1 基础概念

1.3 程序地址

●程序在成为进程前的准备工作

- ❖编辑：形成源文件(**符号地址**)
- ❖编译：形成目标模块(**模块内符号地址解析**)
- ❖链接：由多个目标模块或程序库生成可执行文件
(**模块间符号地址解析**)
- ❖装入：构造PCB，形成进程(**物理地址**)

1.3 程序地址

物理地址和逻辑地址



物理地址（绝对地址）

- 物理内存的地址，内存以字节为单位编址
- 物理地址空间：所有物理地址的集合



逻辑地址（虚拟地址、相对地址）

- 由CPU产生的地址，即程序编译后使用的相对于0字节的地址
- 逻辑地址空间：由程序所生成的所有逻辑地址的集合

1.3 程序地址

物理地址和逻辑地址



地址转换（地址重定位、地址映射）

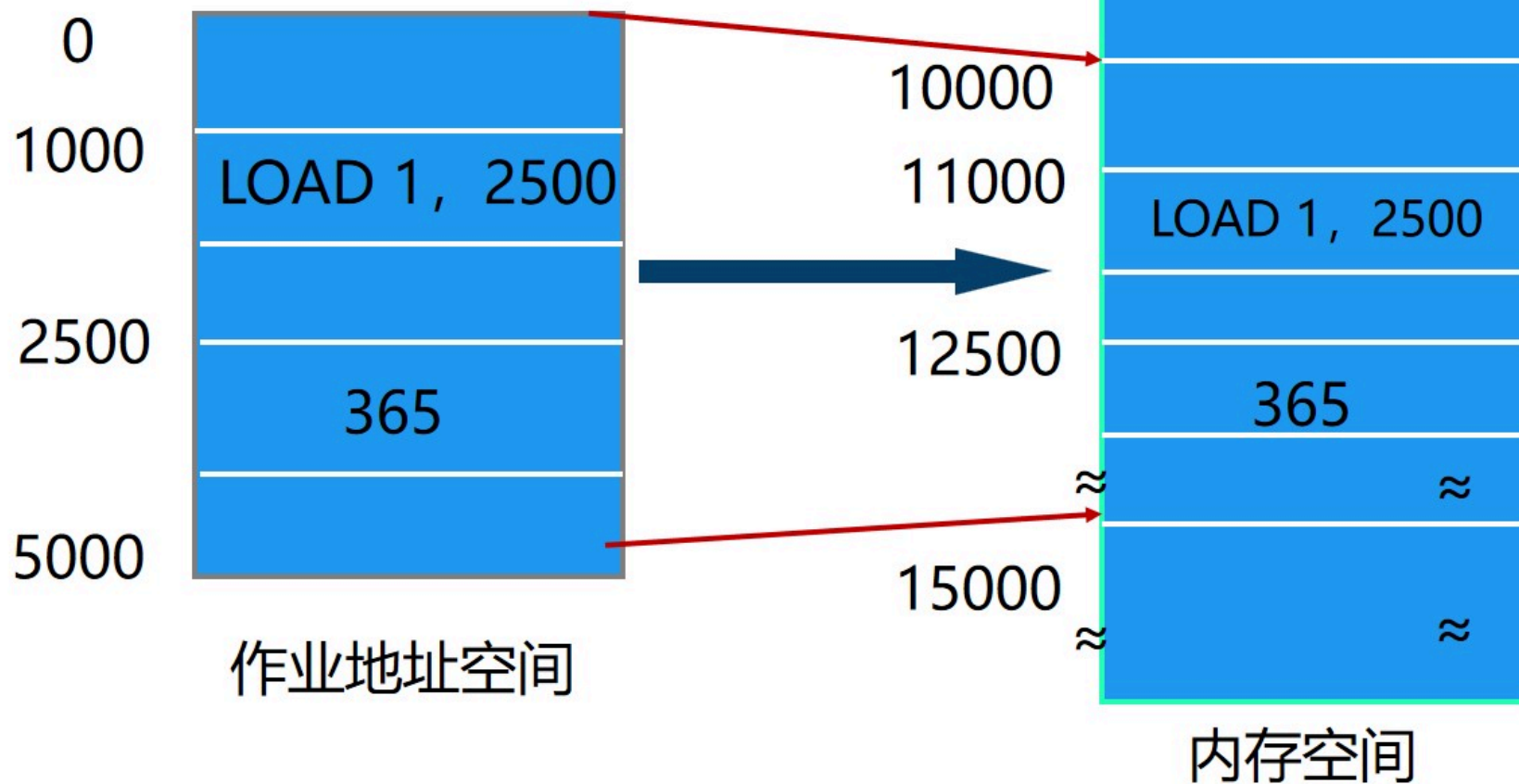
➤ 程序中的逻辑地址转换为物理地址

操作系统内存管理任务之一！

Part 1 基础概念

1.4 程序的装入

作业装入内存示意图



1.4 程序的装入

程序的装入——地址转换时机



绝对装入方式

- 编译时产生的地址使用**绝对地址**
- 程序或数据被修改时，需要重新编译程序



可重定位装入方式

- 编译后的目标模块使用相对地址
- 在**装入**时，完成**重定位**（静态重定位）
- 需硬件支持



动态运行时装入方式

- 编译后的目标模块使用相对地址
- 在**运行**时，完成重定位（动态重定位）

1.5 程序的链接



静态链接

- 在程序运行前，将各目标模块及它们所需的库函数链接成一个完整的装配模块
- 对相对地址进行修改；变换外部调用符号



装入时动态链接

- 在装入内存时，采用边装入边链接的链接方式
- 便于修改和更新
- 便于实现对目标模块的共享



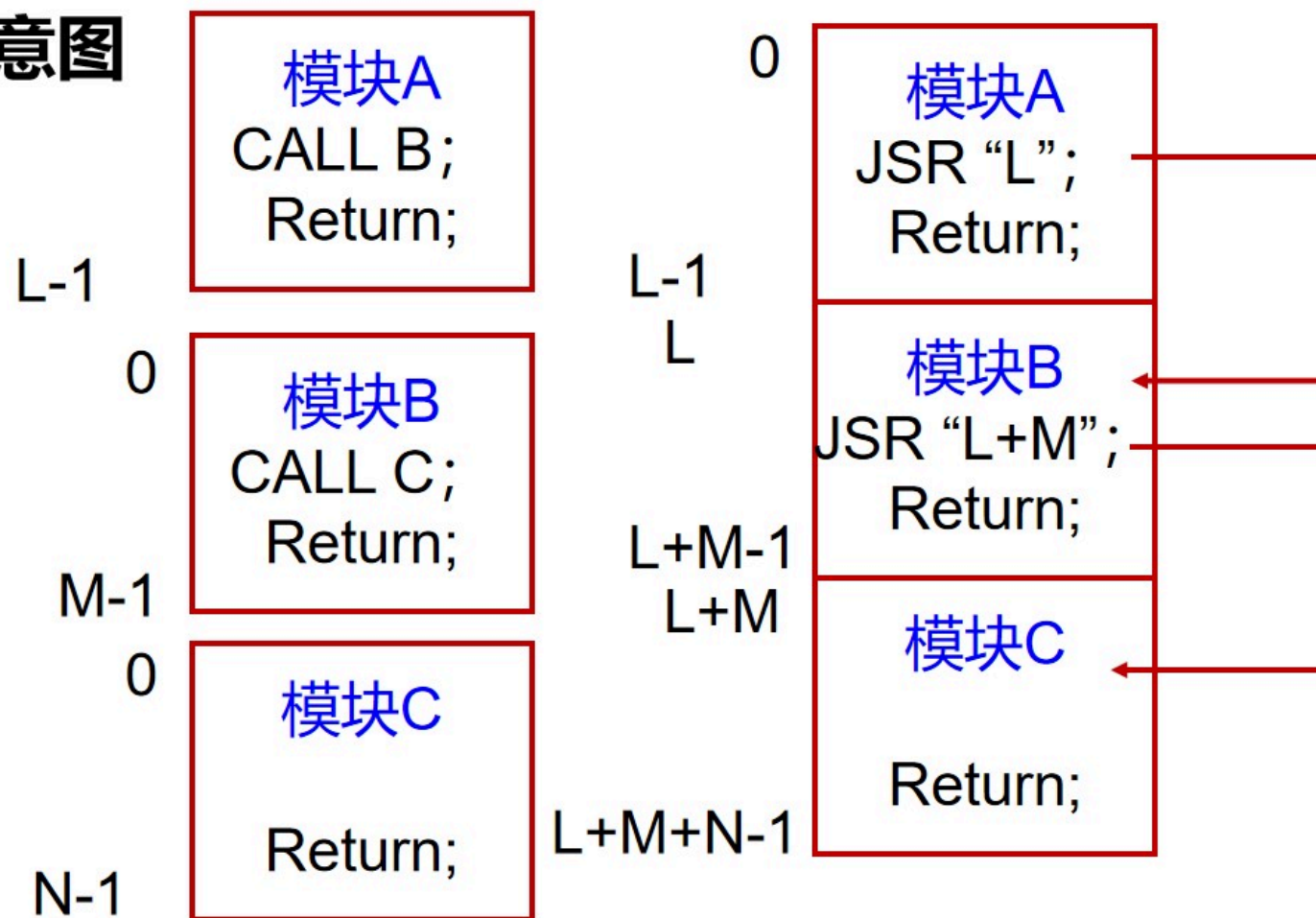
运行时动态链接

- 将某些目标模块的链接推迟到执行时才执行
- 加快装入过程，节省大量的内存空间

Part 1 基础概念

1.5 程序的链接

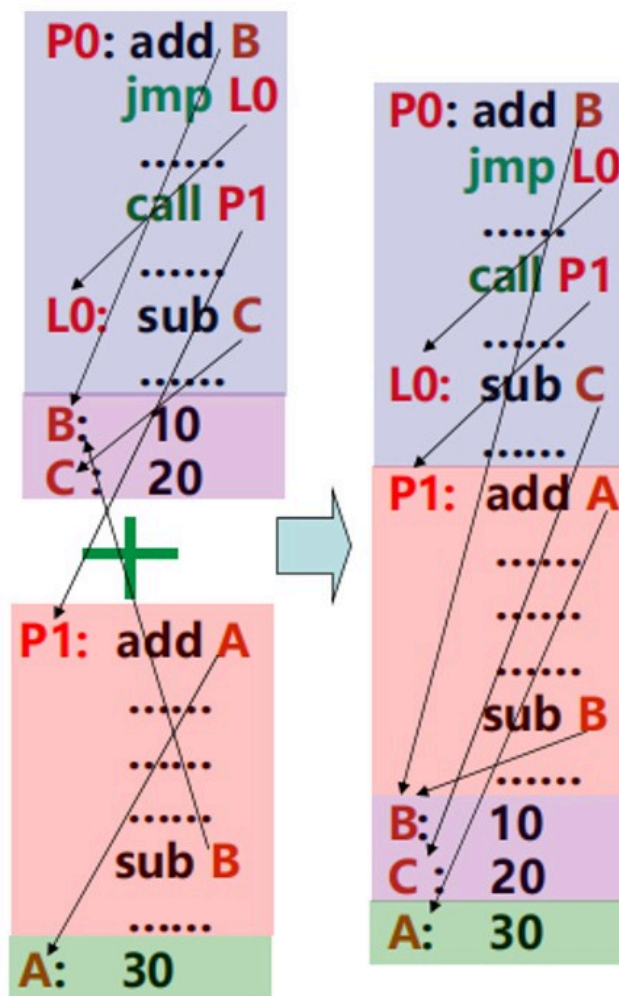
链接示意图



Part 1 基础概念

1.5 程序的链接

链接示意图



各种存储管理方案

(1) 分区存储管理

连续存储

(2) 分页存储管理

(3) 分段存储管理

(4) 段页式存储管理

离散存储

(5) 交换技术和覆盖技术

(6) 虚拟存储管理

虚拟存储

掌握各种存储管理方案的内存分配回收、数据结构、地址转换、存储保护、主要技术、优缺点。

Part 2 连续分配方式



学习目标

- 能够分析单一分区、固定分区、动态分区的管理方式及优缺点
- 掌握操作系统分区管理的数据结构、分区分配算法、回收算法
- 理解并应用分区管理的地址变换方法
- 掌握分区管理的内存扩充技术：交换、覆盖、紧凑

Part 2 连续分配方式

连续分配存储管理方式

连续分配方式：为一个用户程序分配一个连续的内存空间。

分类：



Part 2 连续分配方式

2.1 单一连续分区



分配方式：单道程序环境下，仅装有一道用户程序，整个内存的用户空间由该程序独占。

- 内存分配管理十分简单，内存利用率低
- 用于单用户、单任务OS
- CP/M、MS-DOS、RT11



Part 2 连续分配方式

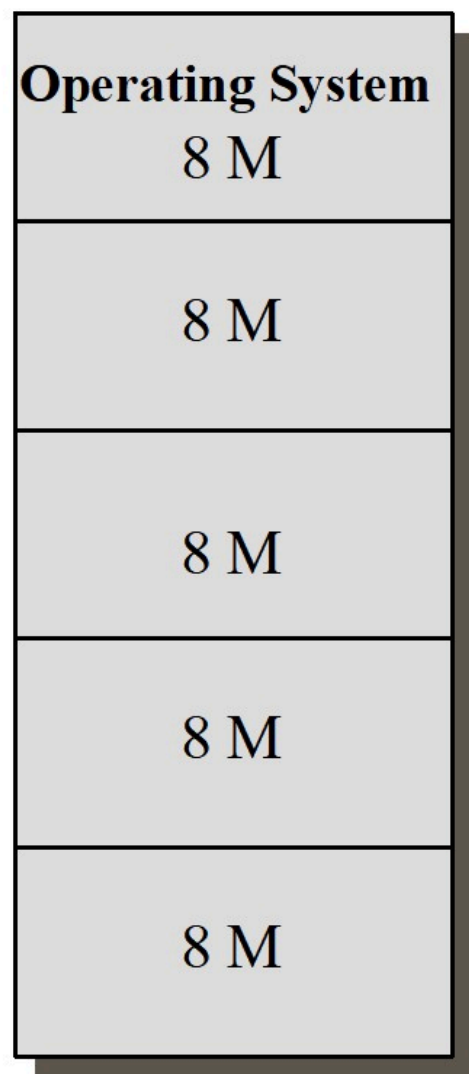
2.2 固定分区

管理方式:

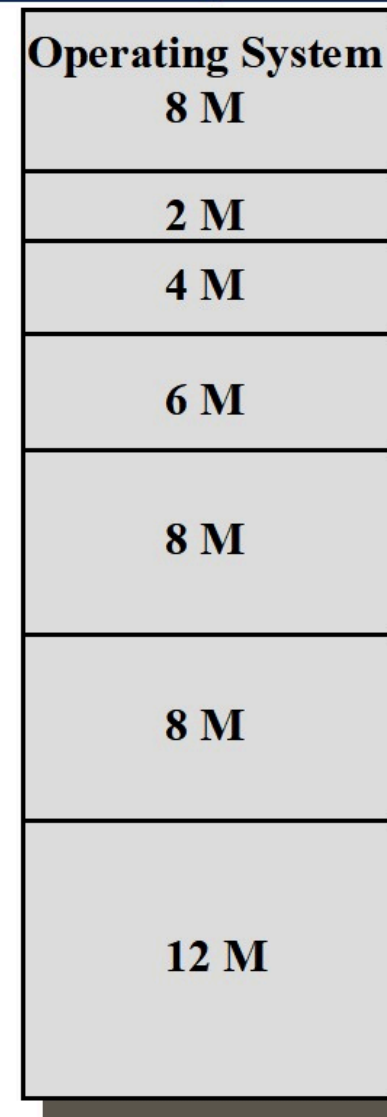
多个分区装可多道程序

分区大小固定

分区个数固定



固定分区(大小相同)



固定分区(多种大小)

Part 2 连续分配方式

2.2 固定分区

内存分配:

分区号	大小(K)	起始地址(K)	状态
1	12	20	已分配
2	32	32	已分配
3	64	64	已分配
4	128	128	未分配

固定分区使用表

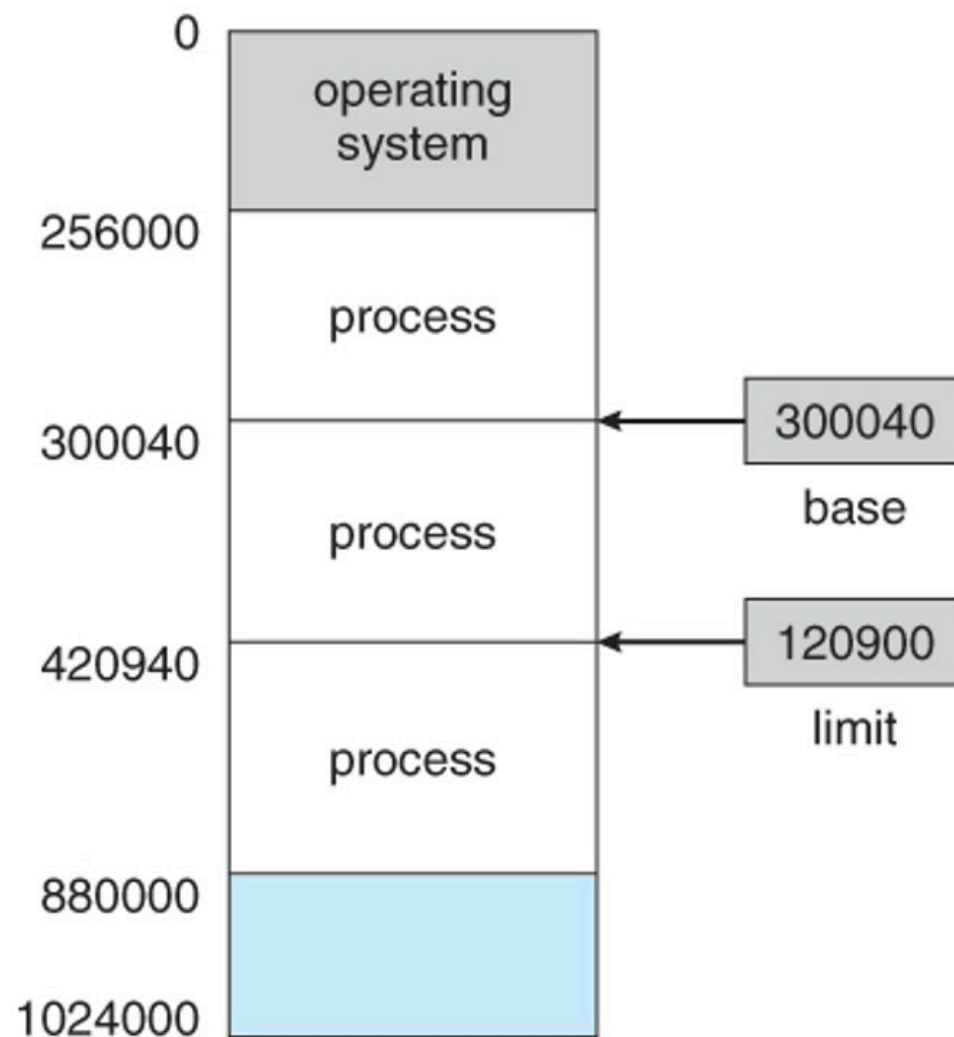
20K	操作系统
32K	作业A
64K	作业B
128K	作业C
256K	

Part 2 连续分配方式

2.2 固定分区

地址重定位与存储保护

- 静态重定位
- 界限寄存器



Part 2 连续分配方式

2.2 固定分区

特点:

优点: 易于实现, 开销小。

问题:

1、可能存在内碎片和外碎片。

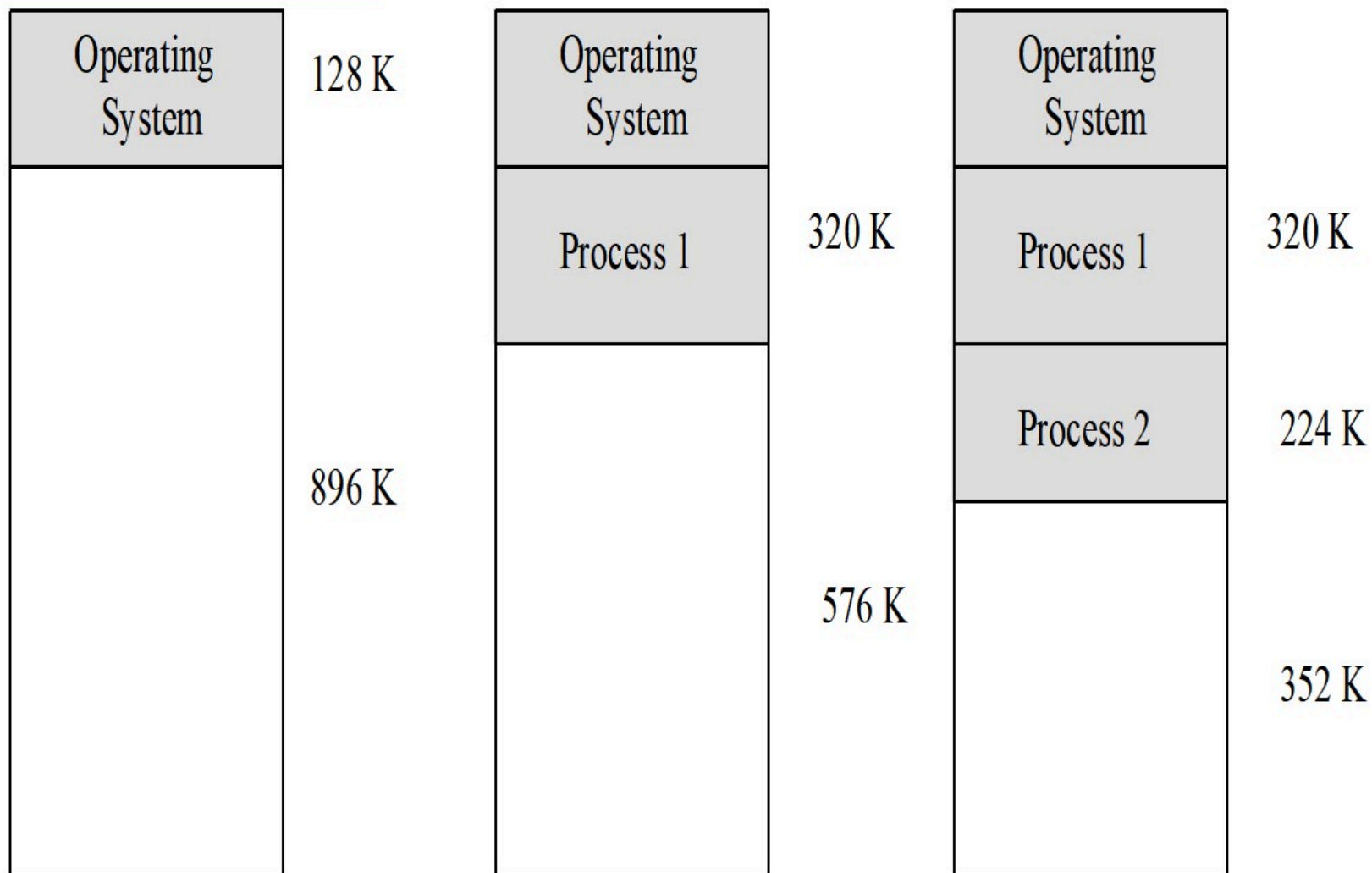
内碎片: 占用分区之内未被利用的空间

外碎片: 占用分区之间难以利用的空闲分区 (通常是小空闲分区)。

2、分区总数固定, 限制了并发执行的程序数目。

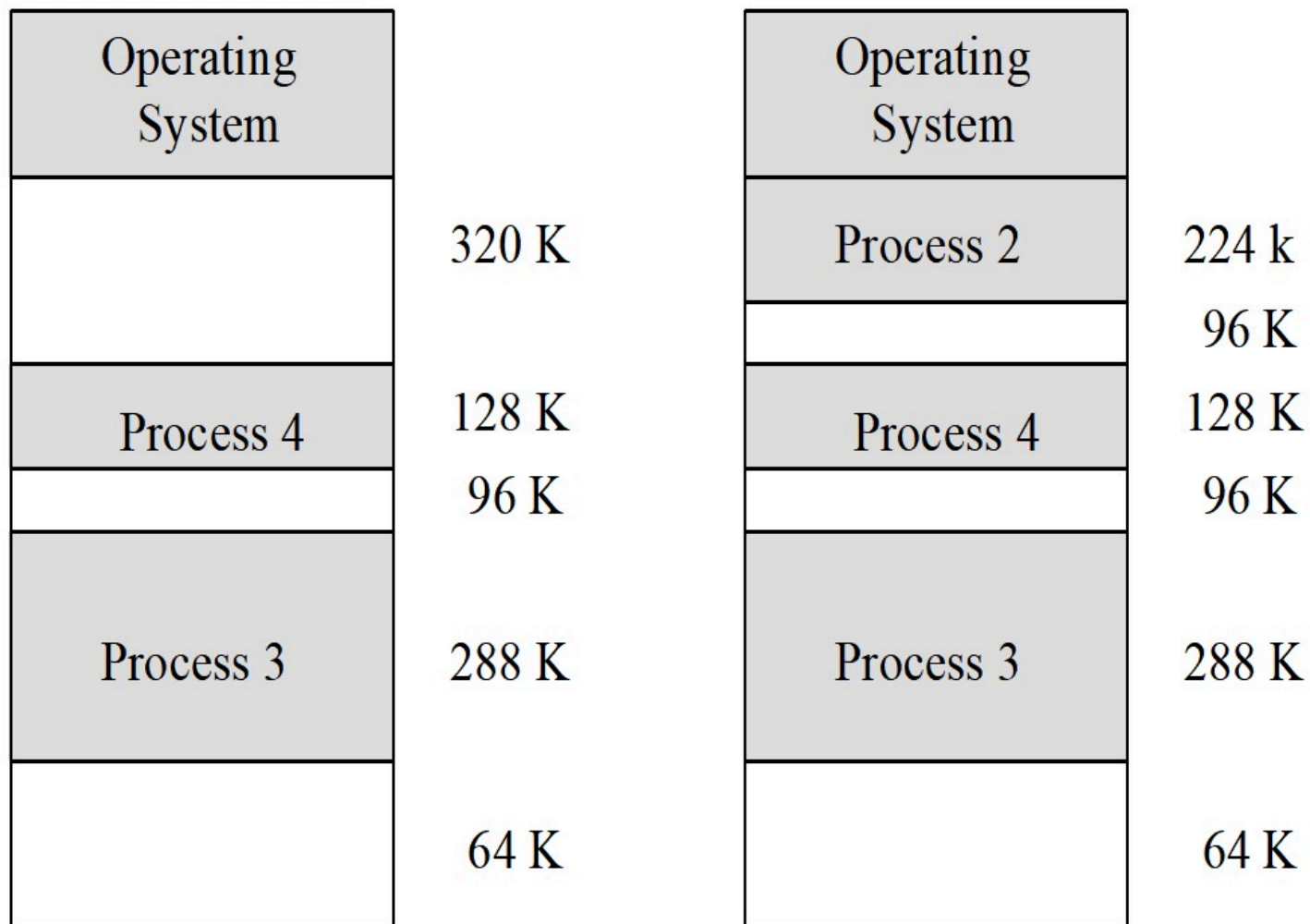
Part 2 连续分配方式

2.3 动态分区



Part 2 连续分配方式

2.3 动态分区



Part 2 连续分配方式

2.3 动态分区

空闲分区表

操作系统
空闲区 (8K)
作业区 (32K)
空闲区 (32K)
作业区 (120K)
空闲区(300K)

1.数据结构

序号	起始地址	尺寸	状态
1	——	——	空
2	28KB	<u>32KB</u>	作业B
3	——	120KB	空
4	92KB	——	作业D
5	——		空

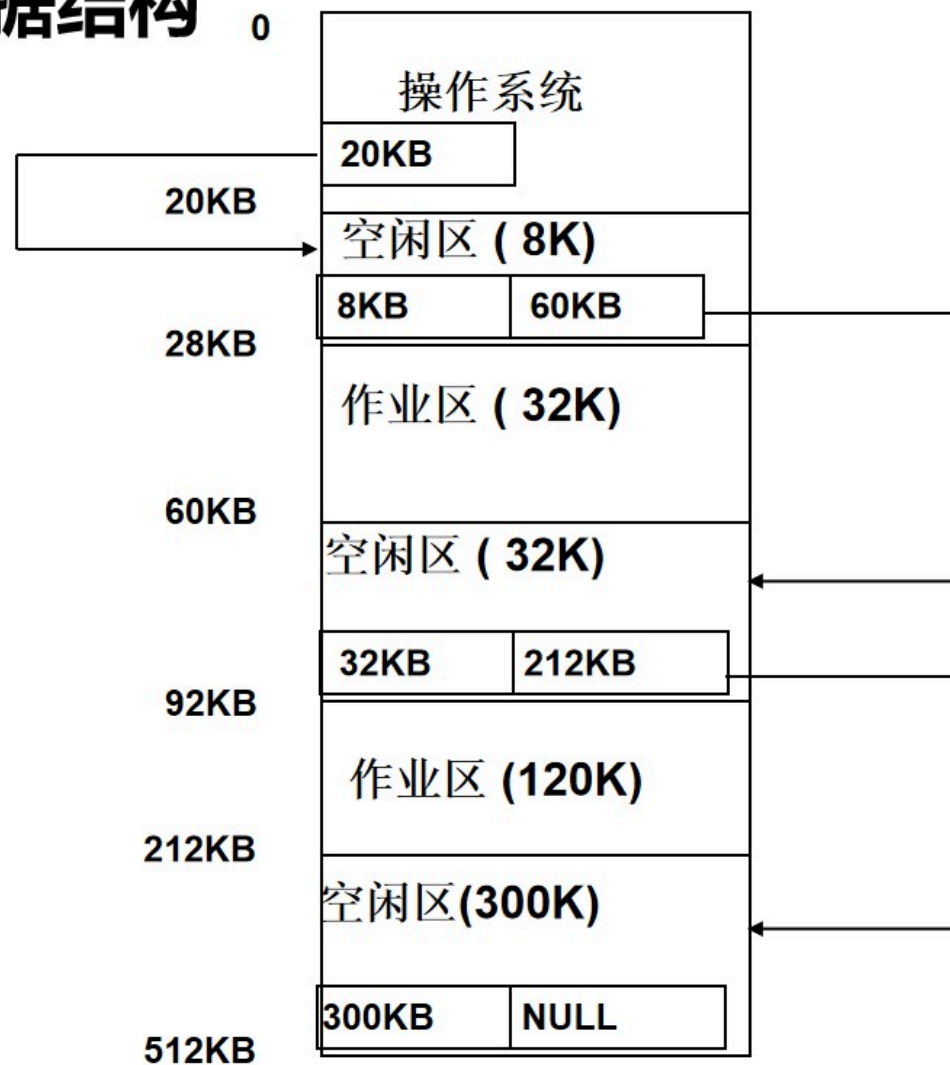
序号	起始地址	尺寸	状态
1	20K	8KB	空闲
2	60KB	32KB	空闲
3	212KB	300KB	空闲
4	——	——	空
5	——	——	空

Part 2 连续分配方式

2.3 动态分区

空闲分区链

1. 数据结构



Part 2 连续分配方式

2.3 动态分区

2.分配算法



基于顺序搜索的动态分区分配算法

- 依次搜索空闲分区链上的空闲分区，寻找一个其大小能够满足要求的分区
- 首次适应算法、循环首次适应算法、最佳适应算法、最坏适应算法



基于索引搜索的动态分区分配算法

- 提高搜索空闲分区的速度，在大、中型系统中采用
- 快速适应算法、伙伴系统和哈希算法

Part 2 连续分配方式

2.3 动态分区 2.分配算法



首次适应算法

- 空闲分区链以地址递增的次序链接
- 从链首开始顺序查找，直到找到一个大小能满足要求的空闲分区为止
- 缺点：低址部分留下许多小碎片



循环首次适应算法

- 从上次找到的空闲分区的下一个空闲分区开始查找，直到找到一个能满足要求的空闲分区
- 空闲分区分布更均匀，减少了查找的开销
- 缺乏大的空闲分区

2.3 动态分区 2.分配算法



最佳适应算法

- 搜索整个序列，找到适合条件的最小的分区进行分配
- 空闲分区按其容量从小到大的顺序链接
- 用最小空间满足要求；但留下许多难以利用的小碎片



最坏适应算法

- 搜索整个序列，寻找最大的分区进行分配
- 空闲分区按其容量从大到小的顺序链接
- 分割后空闲块仍为较大空块；缺乏大的空闲分区

Part 2 连续分配方式

2.3 动态分区

2.分配算法



快速适应算法

- 将空闲分区按其容量大小进行分类，具有相同容量的所有空闲分区设有一个空闲分区链表
- 分配时，根据进程长度，从索引表中寻找到能容纳它的最小空闲分区链表；从链表中取下第一块进行分配

特点

- 优点：不分割分区，不产生碎片，查找效率高
- 缺点：分区归还主存时算法复杂，系统开销较大，存在浪费

Part 2 连续分配方式

2.3 动态分区

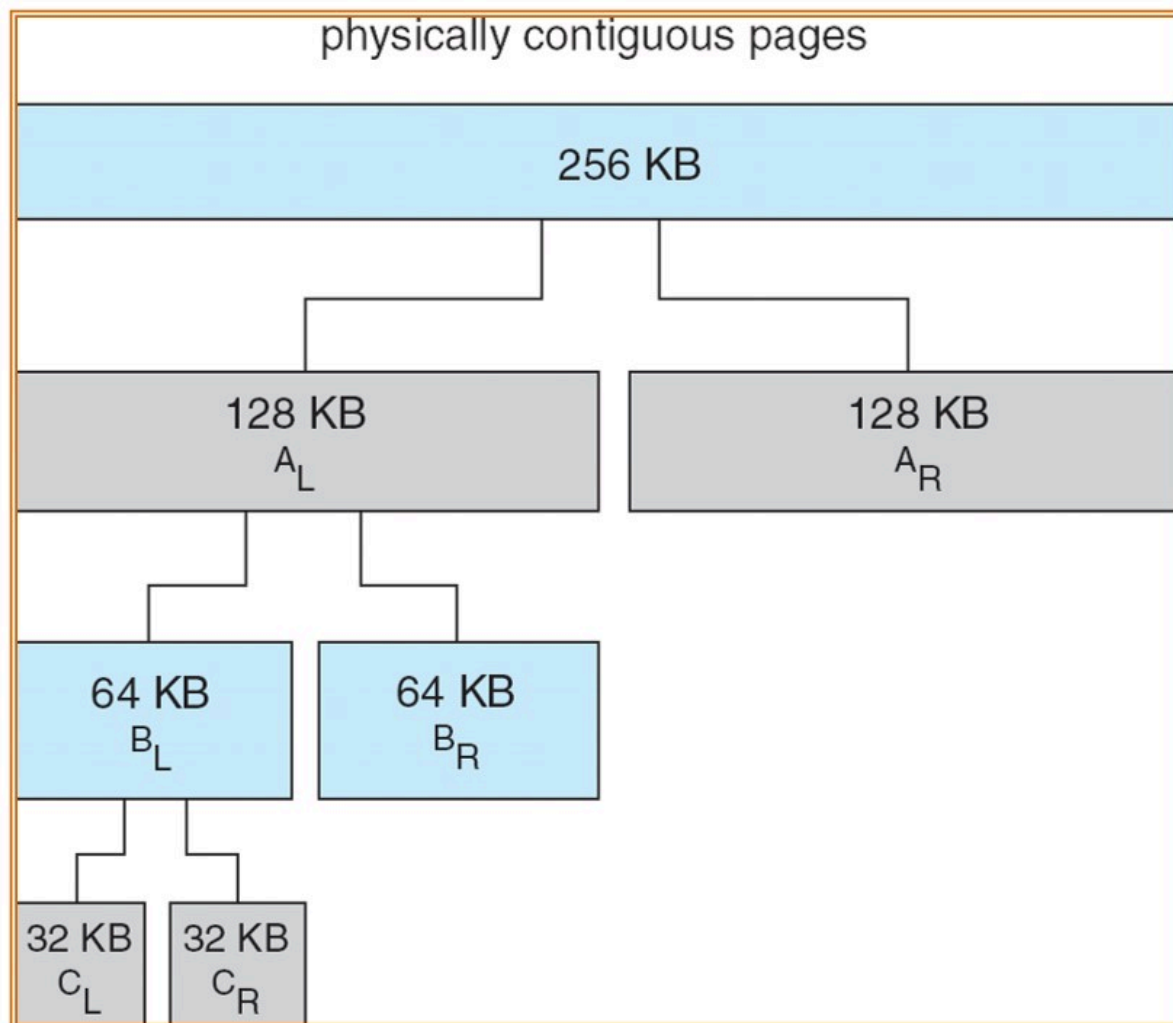


伙伴系统

进程请求

100K, 20K, 50K, 10K

2.分配算法



Part 2 连续分配方式

2.3 动态分区

2.分配算法



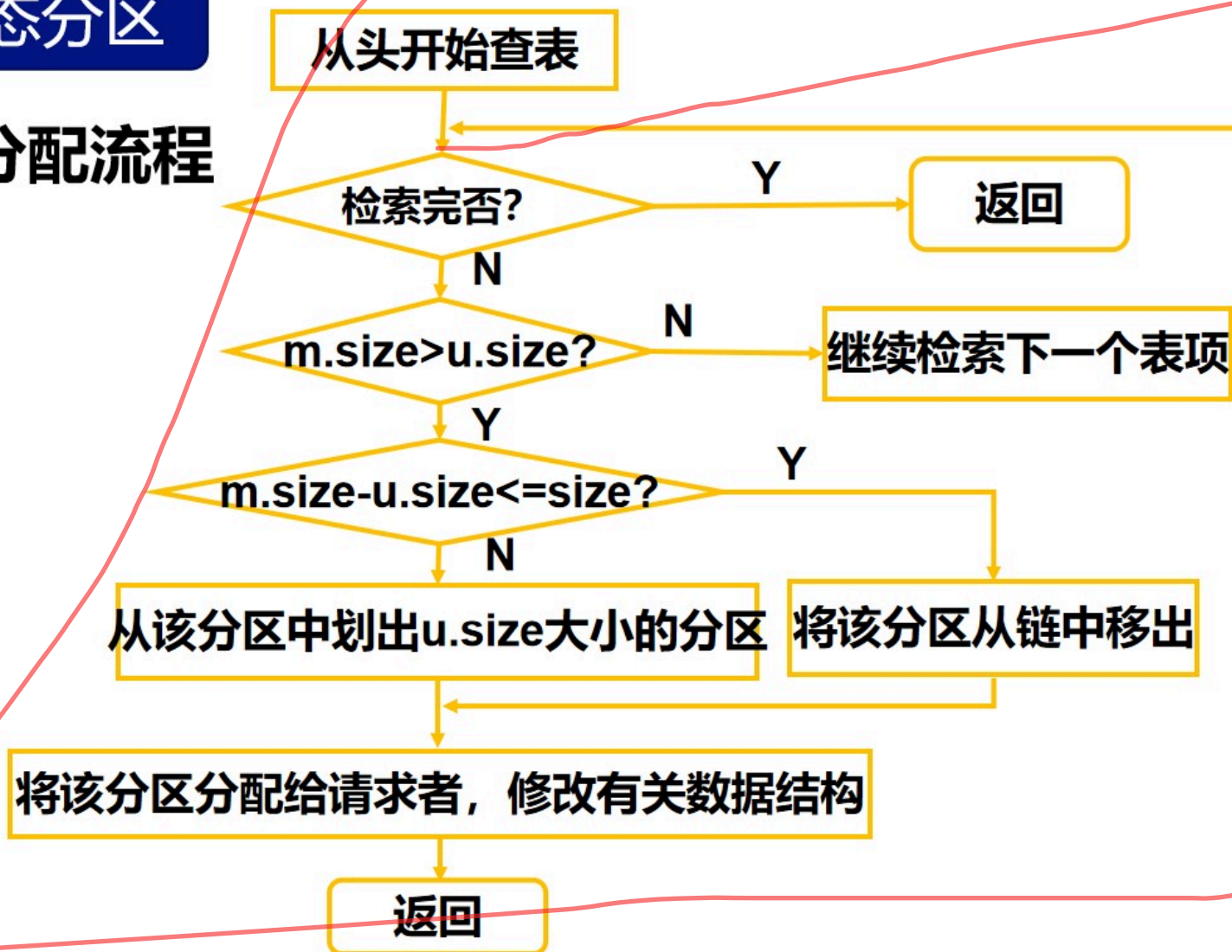
伙伴系统

- 分区大小均为2的k次幂
- 内存按2的幂的大小来分配，即4KB、8KB等
 - 满足要求是以2的幂为单位的
 - 如果请求不为2的幂，则需要调整到下一个更大的2的幂：先计算一个i值，使 $2^{i-1} < n \leq 2^i$ ，在 2^i 的空闲区表中找
 - 当分配需求小于现在可用内存时，当前段就分为两个更小的2的幂段
 - 继续上述操作直到合适的段大小

Part 2 连续分配方式

2.3 动态分区

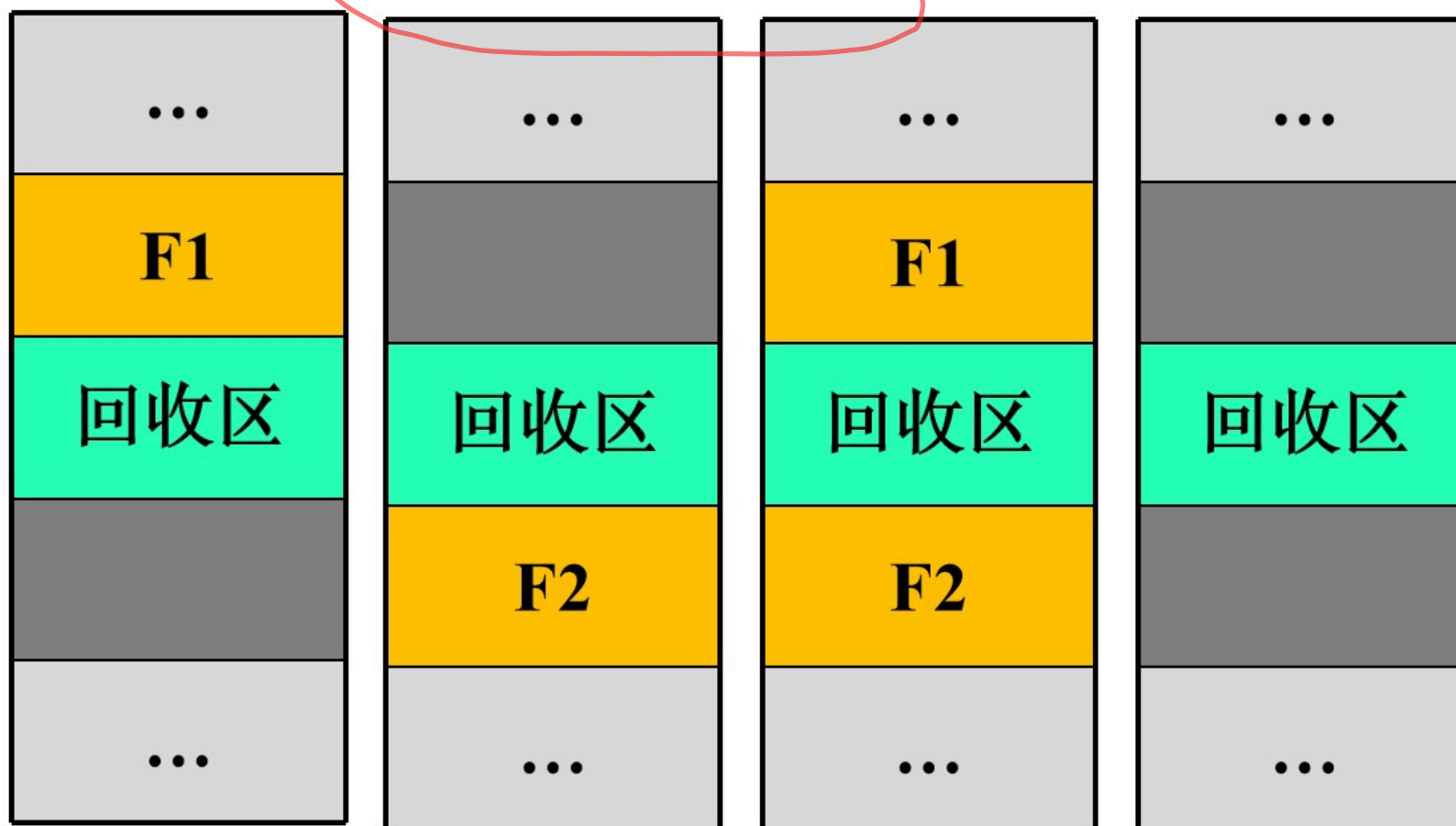
3. 内存分配流程



Part 2 连续分配方式

2.3 动态分区

4. 内存回收

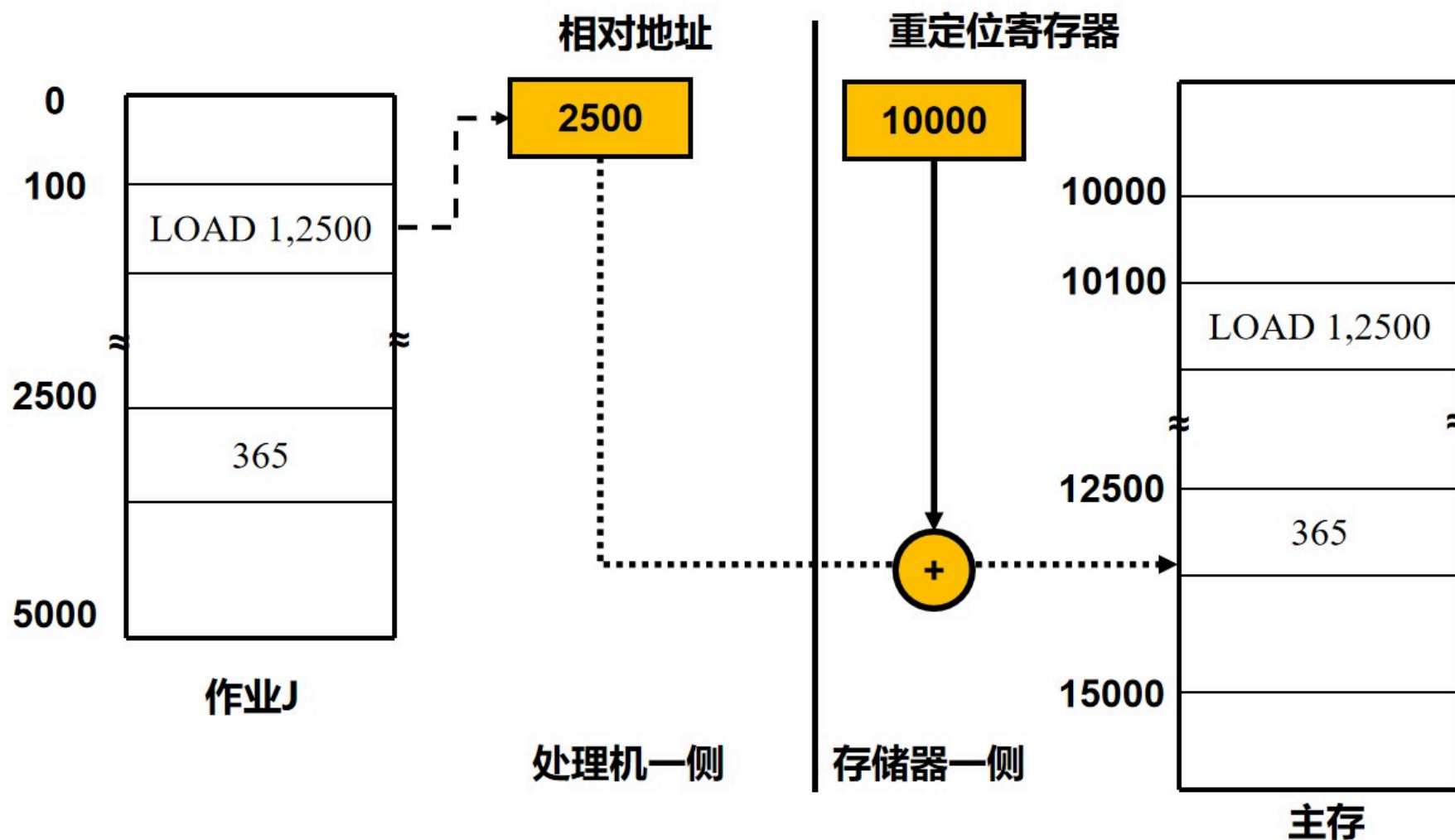


Part 2 连续分配方式

2.3 动态分区

5.地址转换

动态重定位示意图



Part 2 连续分配方式

2.3 动态分区

6. 相关技术

● 主存不足的存储管理技术

❖ 内存紧凑

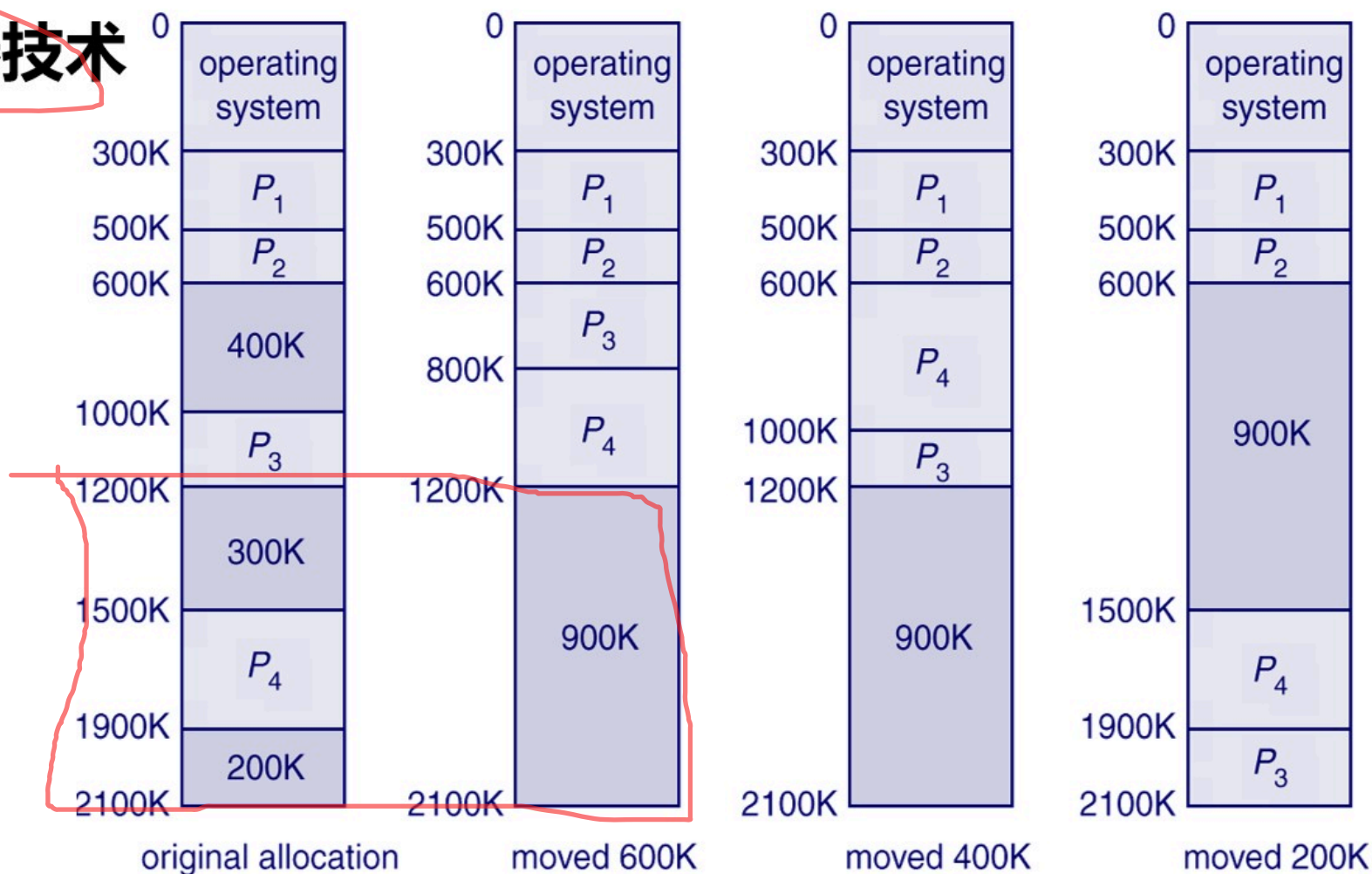
❖ 交换

❖ 覆盖

Part 2 连续分配方式

2.3 动态分区

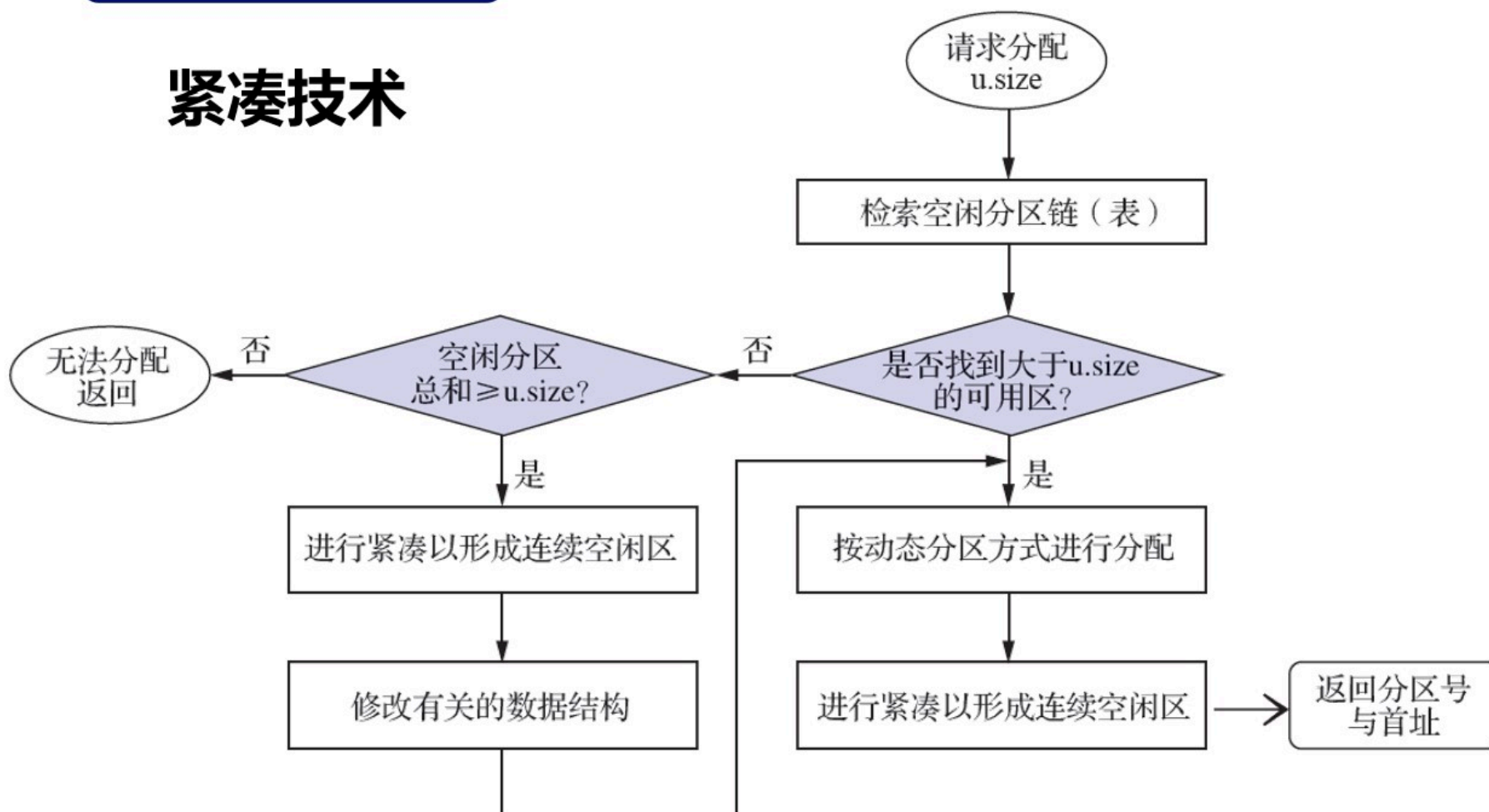
紧凑技术



Part 2 连续分配方式

2.3 动态分区

紧凑技术



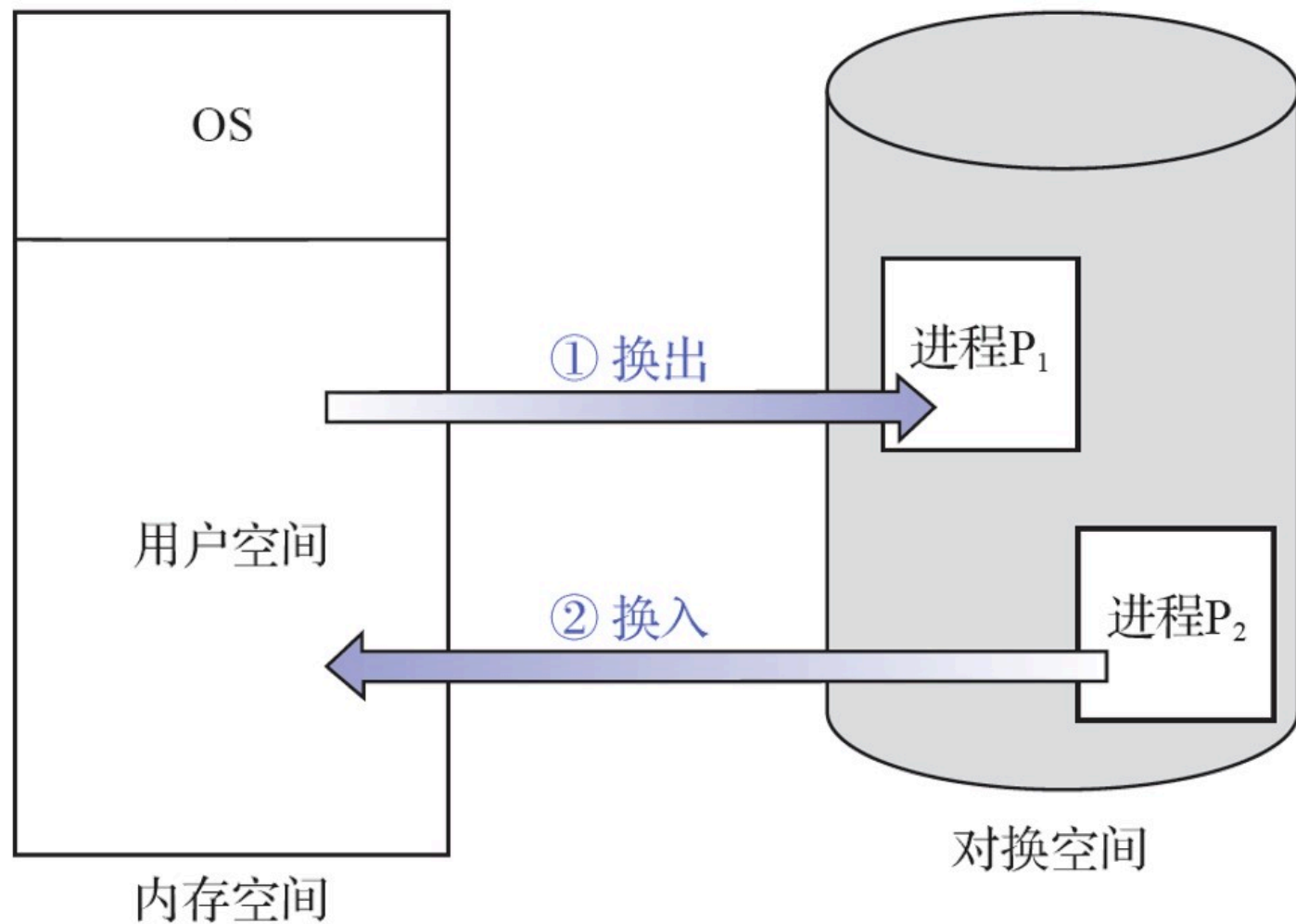
2.4 对换

- 时间换空间技术1 ——对换

最早应用于麻省理工学院的兼容分时系统CTSS中所谓“对换”，是指把内存中暂时不能运行的进程或者暂时不用的程序和数据，调出到外存上，以便腾出足够的内存空间，再把已具备运行条件的进程或进程所需要的程序和数据，调入内存。

Part 2 连续分配方式

2.4 对换



Part 2 连续分配方式

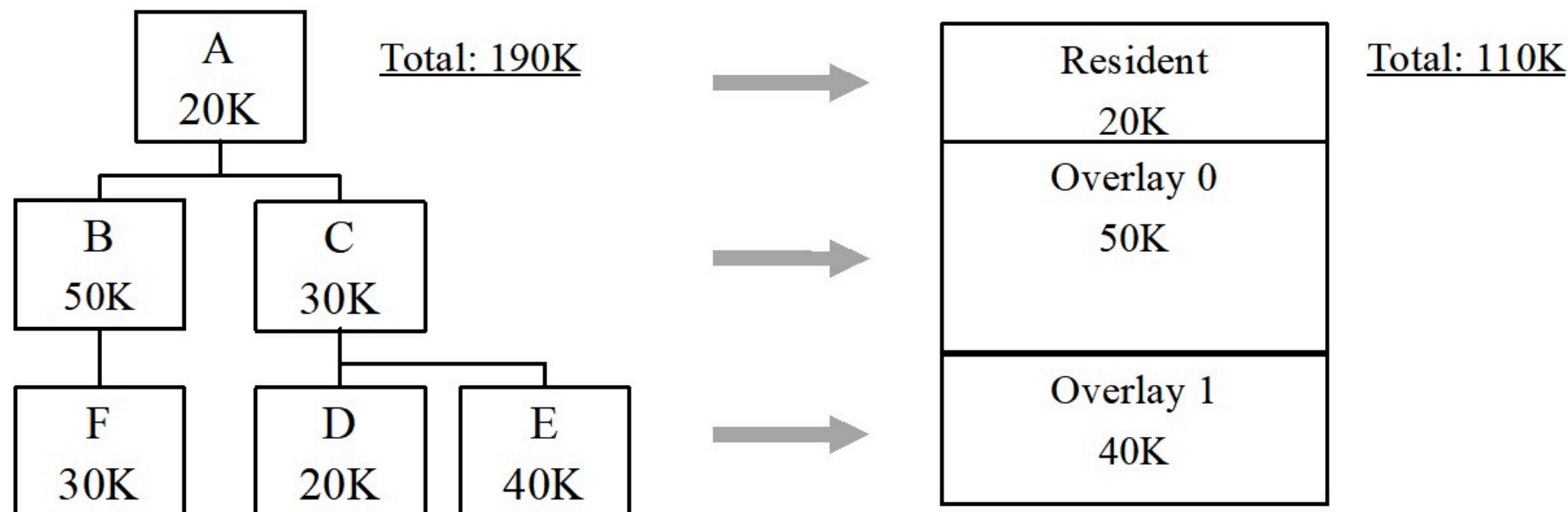
2.5 覆盖

- 时间换空间技术2 ——覆盖

在较小的可用内存中运行较大的程序。常用于多道程序系统，与分区存储管理配合使用。

Part 2 连续分配方式

2.5 覆盖



注：另一种覆盖方法：(100K)

- A(20K)占一个分区：20K；
- B(50K)、D(20K)和E(40K)共用一个分区：50K；
- F(30K)和C(30K)共用一个分区：30K；

下列分区管理方式中，会产生内碎片的是（ ）。

- ☐ A 单一连续分区
- ☒ B 分区大小相等的固定分区
- ☒ C 分区大小不等的固定分区
- ☐ D 可变分区

在可变分区管理时,可用的分区分配算法有 ()。

☒ A 先来先服务

☐ B 最佳适应

☐ C 首次适应

☐ D 伙伴算法

单选题 1分

在可变分区存储管理中,某作业完成后要收回其主存空间,该空间可能与相邻空闲区合并,修改空闲区表,使空闲区始址改变但空闲区数不变的是()情况。

- ☒ A 有上邻空闲区也有下邻空闲区
- ☒ B 有上邻空闲区但无下邻空闲区
- ☐ C 无上邻空闲区但有下邻空闲区
- ☒ D 无上邻空闲区且也无下邻空闲区

单选题 1分

在可变分区存储管理中,某作业完成后要收回其主存空间,该空间可能与相邻空闲区合并,修改表的空闲区大小空闲区始址不改变的是()情况。

- A 有上邻空闲区也有下邻空闲区
- B 有上邻空闲区但无下邻空闲区
- C 无上邻空闲区但有下邻空闲区
- D 无上邻空闲区且也无下邻空闲区

